



大语言模型及应用

Instructor: Xiaohan Ma(马晓寒), Ph.D.

Graduate School of Information and Communication Technology,

Ajou University, Korea

maxiaohan@ajou.ac.kr

课程内容

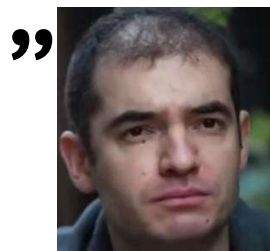
□ 语言模型

- 理解语言模型基本概念和评估方法
- 了解语言模型常见应用
- 掌握基于统计、和基于神经网络的学习方法

□ 表示学习

- 了解词表示的学习概念和意义
- 掌握基于神经网络的词表示学习方法
 - word2vec的基本原理
- 了解不同词表示学习方法的差异
- 了解句子表示的学习概念和意义
- 掌握基于神经网络的句子表示学习方法

“The Next-token prediction is enough for AGI”



Ilya Sutskever
前 OpenAI CSO

TITLE	CITED BY	YEAR
Imagenet classification with deep convolutional neural networks A Krizhevsky, I Sutskever, GE Hinton Advances in neural information processing systems 25	151397 *	2012
Tensorflow: Large-scale machine learning on heterogeneous distributed systems M Abadi, A Agarwal, P Barham, E Brevdo, Z Chen, C Citro, GS Corrado, ... arXiv preprint arXiv:1603.04467	51600 *	2016
Dropout: a simple way to prevent neural networks from overfitting N Srivastava, G Hinton, A Krizhevsky, I Sutskever, R Salakhutdinov The journal of machine learning research 15 (1), 1929-1958	49111	2014
Distributed representations of words and phrases and their compositionality T Mikolov, I Sutskever, K Chen, GS Corrado, J Dean Advances in neural information processing systems 26	42827	2013
Sequence to sequence learning with neural networks I Sutskever, O Vinyals, QV Le Advances in neural information processing systems 27	25262	2014
Language models are few-shot learners T Brown, B Mann, N Ryder, M Subbiah, JD Kaplan, P Dhariwal, ... Advances in neural information processing systems 33, 1877-1901	21815	2020
Mastering the game of Go with deep neural networks and tree search D Silver, A Huang, CJ Maddison, A Guez, L Sifre, G Van Den Driessche, ... nature 529 (7587), 484-489	18239	2016
Intriguing properties of neural networks C Szegedy, W Zaremba, I Sutskever, J Bruna, D Erhan, I Goodfellow, ... arXiv preprint arXiv:1312.6199	15836	2013
Learning transferable visual models from natural language supervision A Radford, JW Kim, C Hallacy, A Ramesh, G Goh, S Agarwal, G Sastry, ... International conference on machine learning, 8748-8763	13231	2021
Improving neural networks by preventing co-adaptation of feature detectors GE Hinton, N Srivastava, A Krizhevsky, I Sutskever, RR Salakhutdinov arXiv preprint arXiv:1207.0580	10997	2012
Language models are unsupervised multitask learners A Radford, J Wu, R Child, D Luan, D Amodei, I Sutskever OpenAI blog 1 (8), 9	8929	2019
Improving language understanding by generative pre-training A Radford, K Narasimhan, T Salimans, I Sutskever	8363	2018
Infogan: Interpretable representation learning by information maximizing generative adversarial nets X Chen, Y Duan, R Houthoofd, J Schulman, I Sutskever, P Abbeel Advances in neural information processing systems 29	6019 *	2016

“The Next-token prediction is enough for AGI”



Ilya Sutskever
前 OpenAI CSO

Predicting the next token well means that **you understand the underlying reality that led to the creation of that token.**

It's the statistics but what is statistics? In order to understand those statistics to compress them, you need to **understand what is it about the world that creates those statistics**

语言模型

李明回家的路上被小狗咬了

小狗回家的路上被李明咬了

语言模型

李明回家的路上被小狗咬了

更容易发生

小狗回家的路上被李明咬了

语言模型

今天天气太热，我回家路上吃了__

包子

烤红薯

冰淇淋

语言模型

今天天气太热，我回家路上吃了——

包子
烤红薯
冰淇淋

更容易发生

语言模型

李明回家的路上被小狗咬了

小狗回家的路上被李明咬了

$P(\text{李明回家的路上被小狗咬了}) > P(\text{小狗回家的路上被李明咬了})$

语言模型

今天天气太热，我回家路上吃了——

包子

烤红薯

冰淇淋

更容易发生

$P(\text{冰淇淋} | \text{今天天气太热，我回家路上吃了}) > P(\text{烤红薯} | \text{今天天气太热，我回家路上吃了})$

语言模型

- 给每一个句子都赋予其出现的概率。以 $w_1, w_2, \dots, w_n$ 这 n 个词组成的句子为例，它出现的概率为

$$P(w_1 w_2 \dots w_n) = P(w_1) P(w_2 | w_1) \dots P(w_n | w_1 w_2 \dots w_{n-1})$$

来衡量句子符合自然语言的语法和语义规则的置信度

语言模型

- 给每一个句子都赋予其出现的概率。以 $w_1, w_2, \dots, w_n$ 这 n 个词组成的句子为例，它出现的概率为

$$P(w_1 w_2 \dots w_n) = P(w_1) P(w_2 | w_1) \dots P(w_n | w_1 w_2 \dots w_{n-1})$$

- 语言模型的基本功能
 - 判别：判断一段文本是否符合一种语言的语法和语义规则

语言模型

- 给每一个句子都赋予其出现的概率。以 $w_1, w_2, \dots, w_n$, 这 n 个词组成的句子为例, 它出现的概率为

$$P(w_1 w_2 \dots w_n) = P(w_1) P(w_2 | w_1) \dots P(w_n | w_1 w_2 \dots w_{n-1})$$

- 语言模型的基本功能

- 判别: 判断一段文本是否符合一种语言的语法和语义规则

李明回家的路上被小狗咬了

概率更大

小狗回家的路上被李明咬了

She went to Shanghai yesterday

概率更大

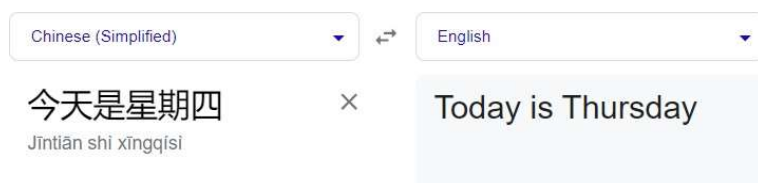
She go to Shanghai yesterday

语言模型

- 给每一个句子都赋予其出现的概率。以 $w_1, w_2, \dots, w_n$, 这 n 个词组成的句子为例, 它出现的概率为

$$P(w_1 w_2 \dots w_n) = P(w_1) P(w_2 | w_1) \dots P(w_n | w_1 w_2 \dots w_{n-1})$$

- 语言模型的基本功能
 - 判别: 判断一段文本是否符合一种语言的语法和语义规则
 - 生成: 生成符合一种语言语法或语义规则的文本



机器翻译

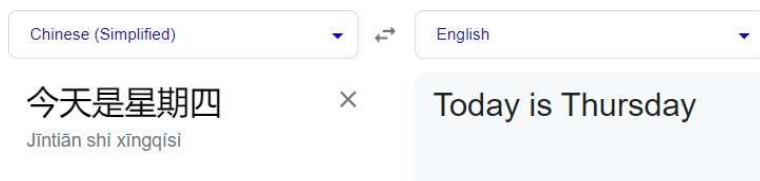
语言模型

- 给每一个句子都赋予其出现的概率。以 $w_1, w_2, \dots, w_n$ 这 n 个词组成的句子为例，它出现的概率为

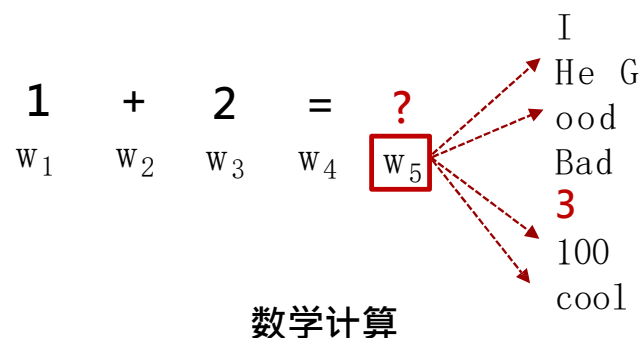
$$P(w_1 w_2 \dots w_n) = P(w_1) P(w_2 | w_1) \dots P(w_n | w_1 w_2 \dots w_{n-1})$$

- 语言模型的基本功能

- 判别：判断一段文本是否符合一种语言的语法和语义规则
- 生成：生成符合一种语言语法或语义规则的文本



机器翻译



数学计算

语言模型

- 给每一个句子都赋予其出现的概率。以 $w_1, w_2, \dots, w_n$, 这 n 个词组成的句子为例, 它出现的概率为

$$P(w_1 w_2 \dots w_n) = P(w_1) P(w_2 | w_1) \dots P(w_n | w_1 w_2 \dots w_{n-1})$$

- 语言模型的基本功能
 - 判别: 判断一段文本是否符合一种语言的语法和语义规则
 - 生成: 生成符合一种语言语法或语义规则的文本
 - 任务: 信息压缩、构建世界模型

如何学习一个语言模型?

$$S = w_1 \cdots w_T \longrightarrow \boxed{f(w_1 \cdots w_T | \theta)} \longrightarrow P(w_1 w_2 \dots w_n)$$

如何学习一个语言模型?

$$S = w_1 \cdots w_T \longrightarrow \boxed{f(w_1 \cdots w_T | \theta)} \longrightarrow P(w_1 w_2 \dots w_n)$$

- 语言模型看到的不是“句子”，而是“token序列”
- token 不一定是字或词

语言模型 = 一个带参数 θ 的函数

语言模型最终的目标是：
给整个句子一个概率。

方法1：统计世界上所有的句子

$$S = w_1 \cdots w_T \longrightarrow \boxed{f(w_1 \cdots w_T | \theta)} \longrightarrow P(w_1 w_2 \dots w_n)$$

$$D = \{S_i\}_{i=1}^{\boxed{N}} \quad \text{所有可能组合的句子可能}$$
$$f(\cdot) = \frac{\text{count}(s)}{N}$$

方法1：统计世界上所有的句子

$$S = w_1 \cdots w_T \longrightarrow \boxed{f(w_1 \cdots w_T | \theta)} \longrightarrow P(w_1 w_2 \dots w_n)$$

$$D = \{S_i\}_{i=1}^{\boxed{N}}$$

所有可能组合的句子可能

$$f(\cdot) = \frac{\text{count}(s)}{N}$$

不可计算！

方法2：统计世界上所有的单词和其组合

$$P(S = w_{1:T}) = P(w_1, \dots, w_T)$$

$$= P(w_1)P(w_2|w_1)P(w_3|w_1w_2) \cdots P(w_T|w_{1:(T-1)})$$

概率公式

$$= P(w_1) \prod_{t=2}^T P(w_t|w_{1:(t-1)})$$

$$= \prod_{t=1}^T P(w_t|w_{1:(t-1)}),$$

句子中每个词 w_t 在给定前面词序列 $w_{1:(t-1)}$ 时的条件概率

方法2：统计世界上所有的单词和其组合

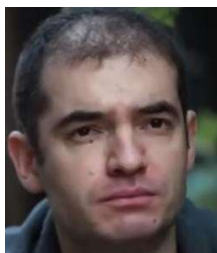
$$P(S = w_{1:T}) = P(w_1, \dots, w_T)$$

$$= P(w_1)P(w_2|w_1)P(w_3|w_1w_2) \cdots P(w_T|w_{1:(T-1)})$$
 概率公式

$$= P(w_1) \prod_{t=2}^T P(w_t|w_{1:(t-1)})$$

$$= \prod_{t=1}^T P(w_t|w_{1:(t-1)}),$$

句子中每个词 w_t 在给定前面词序列 $w_{1:(t-1)}$ 时的条件概率



Ilya Sutskever
OpenAI CSO

语言模型 = 通过预测下一个词来间接建模整个句子的概率。

The Next-token prediction is enough for AGI

图例

$P(\mathbf{I}) =$

$P(\mathbf{I})$
└──┘

Probability of $\mathbf{I}$

方法2：统计世界上所有的单词和其组合

$$P(S = w_{1:T}) = P(w_1, \dots, w_T)$$

$$= P(w_1)P(w_2|w_1)P(w_3|w_1w_2) \cdots P(w_T|w_{1:(T-1)})$$

概率公式

$$= P(w_1) \prod_{t=2}^T P(w_t|w_{1:(t-1)})$$

$$= \prod_{t=1}^T P(w_t|w_{1:(t-1)}),$$

句子中每个词 w_t 在给定前面词序列 $w_{1:(t-1)}$ 时的条件概率

$$P(w_t|w_{1:(t-1)}) = P(w_t|w_{(t-n+1):(t-1)})$$

马尔可夫假设：一个词的概率只依赖于其前面的 $n-1$ 个词

N元语言模型

三元语言模型

Before

$P(\text{I saw a cat on a mat}) =$

$P(\text{I})$

- $P(\text{saw} | \text{I})$
- $P(\text{a} | \text{I saw})$
- $P(\text{cat} | \text{I saw a})$
- $P(\text{on} | \text{I saw a cat})$
- $P(\text{a} | \text{I saw a cat on})$
- $P(\text{mat} | \text{I saw a cat on a})$

After (3-gram)

$P(\text{I saw a cat on a mat}) =$

$P(\text{I})$

- | | | |
|---|---|----------------------------------|
| • $P(\text{saw} \text{I})$ | → | $P(\text{I})$ |
| • $P(\text{a} \text{I saw})$ | → | • $P(\text{saw} \text{I})$ |
| • $P(\text{cat} \text{I saw a})$ | → | • $P(\text{a} \text{I saw})$ |
| • $P(\text{on} \text{I saw a cat})$ | → | • $P(\text{cat} \text{saw a})$ |
| • $P(\text{a} \text{I saw a cat on})$ | → | • $P(\text{on} \text{a cat})$ |
| • $P(\text{mat} \text{I saw a cat on a})$ | → | • $P(\text{a} \text{cat on})$ |
| | | • $P(\text{mat} \text{on a})$ |
- ignore use



统计语言模型

□ 一元 语言模型

$$P(w_1, \dots, w_T) = \prod_{t=1}^T P(w_t)$$

□ 二元 语言模型

$$P(w_1, \dots, w_T) = \prod_{t=1}^T P(w_t | w_{t-1})$$

□ 三元 语言模型

$$P(w_1, \dots, w_T) = \prod_{t=1}^T P(w_t | w_{t-2}, w_{t-1})$$

□ N元 语言模型

$$P(w_1, \dots, w_T) = \prod_{t=1}^T P(w_t | w_{t-n+1}, \dots, w_{t-1})$$

统计语言模型

□ 一元 语言模型

某个词出现的比例。

条件概率=条件出现次数/联合出现次数

$$P(w) = \frac{\text{count}(w)}{\text{所有词的总次数}}$$

概率 = 出现次数 / 总次数

□ 二元 语言模型

在前一个词已经出现的情况下,
当前词出现的比例。

$$P(w_t | w_{t-1}) = \frac{\text{count}(w_{t-1}, w_t)}{\text{count}(w_{t-1})}$$

□ 三元 语言模型

在前两个词已经出现的情况下,
当前词出现的比例。

$$P(w_t | w_{t-2}, w_{t-1}) = \frac{\text{count}(w_{t-2}, w_{t-1}, w_t)}{\text{count}(w_{t-2}, w_{t-1})}$$

□ N元 语言模型

在上下文 c 已经出现的情况下,
当前词出现的比例。

$$P(w_t | c) = \frac{\text{count}(c, w_t)}{\text{count}(c)} \quad \text{其中 :}$$

• c 是上下文 (前 n-1 个词)

例子: 实现一个二元语言模型

语料库

The swift fox jumps over the lazy dog.

The swift river flows under the ancient bridge.

The swift breeze cools the warm summer evening.

如何实现？

语言模型 $P(w_1, \dots, w_T) = \prod_{t=1}^T P(w_t | w_{t-1})$

$$P(w_t | w_{t-1}) = \frac{\text{count}(w_{t-1}, w_t)}{\text{count}(w_{t-1})}$$

例子: 实现一个二元语言模型

语料库

The swift fox jumps over the lazy dog.

The swift river flows under the ancient bridge.

The swift breeze cools the warm summer evening.

如何实现？

统计语言模型 – “数数” (Counting)

语言模型

$$P(w_1, \dots, w_T) = \prod_{t=1}^T P(w_t | w_{t-1})$$

$$P(w_t | w_{t-1}) = \frac{\text{count}(w_{t-1}, w_t)}{\text{count}(w_{t-1})}$$

例子: 实现一个二元语言模型

语料库

The swift fox jumps over the lazy dog.

The swift river flows under the ancient bridge.

The swift breeze cools the warm summer evening.

语言模型

$$P(w_1, \dots, w_T) = \prod_{t=1}^T P(w_t | w_{t-1})$$

$$P(w_t | w_{t-1}) = \frac{\text{count}(w_{t-1}, w_t)}{\text{count}(w_{t-1})}$$

统计量

count(the swift) = 3

...

Previous Word	Next Word	Probability
<s>	the	1.000
ancient	bridge	1.000
breeze	cools	1.000
bridge	</s>	1.000
cools	the	1.000
dog	</s>	1.000
evening	</s>	1.000
flows	under	1.000
fox	jumps	1.000
jumps	over	1.000
lazy	dog	1.000
over	the	1.000
river	flows	1.000
summer	evening	1.000
swift	breeze	0.333
swift	fox	0.333
swift	river	0.333
the	ancient	0.167
the	lazy	0.167
the	swift	0.500
the	warm	0.167
under	the	1.000
warm	summer	1.000

例子: 实现一个二元语言模型

语料库

The swift fox jumps over the lazy dog.

The swift river flows under the ancient bridge.

The swift breeze cools the warm summer evening.

语言模型

$$P(w_1, \dots, w_T) = \prod_{t=1}^T P(w_t | w_{t-1})$$

$$P(w_t | w_{t-1}) = \frac{\text{count}(w_{t-1}, w_t)}{\text{count}(w_{t-1})}$$

参数估计

$P(\text{swift} | \text{the}) = 3/6$

$P(\text{over} | \text{jumps}) = 1$

Previous Word	Next Word	Probability
<s>	the	1.000
ancient	bridge	1.000
breeze	cools	1.000
bridge	</s>	1.000
cools	the	1.000
dog	</s>	1.000
evening	</s>	1.000
flows	under	1.000
fox	jumps	1.000
jumps	over	1.000
lazy	dog	1.000
over	the	1.000
river	flows	1.000
summer	evening	1.000
swift	breeze	0.333
swift	fox	0.333
swift	river	0.333
the	ancient	0.167
the	lazy	0.167
the	swift	0.500
the	warm	0.167
under	the	1.000
warm	summer	1.000

例子: 实现一个二元语言模型

语料库

The swift fox jumps over the lazy dog.

The swift river flows under the ancient bridge.

The swift breeze cools the warm summer evening.

语言模型

$$P(w_1, \dots, w_T) = \prod_{t=1}^T P(w_t | w_{t-1})$$

$$P(w_t | w_{t-1}) = \frac{\text{count}(w_{t-1}, w_t)}{\text{count}(w_{t-1})}$$

p(The swift fox)?

Previous Word	Next Word	Probability
<s>	the	1.000
ancient	bridge	1.000
breeze	cools	1.000
bridge	</s>	1.000
cools	the	1.000
dog	</s>	1.000
evening	</s>	1.000
flows	under	1.000
fox	jumps	1.000
jumps	over	1.000
lazy	dog	1.000
over	the	1.000
river	flows	1.000
summer	evening	1.000
swift	breeze	0.333
swift	fox	0.333
swift	river	0.333
the	ancient	0.167
the	lazy	0.167
the	swift	0.500
the	warm	0.167
under	the	1.000
warm	summer	1.000

例子: 实现一个二元语言模型

语料库

The swift fox jumps over the lazy dog.

The swift river flows under the ancient bridge.

The swift breeze cools the warm summer evening.

语言模型

$$P(w_1, \dots, w_T) = \prod_{t=1}^T P(w_t | w_{t-1})$$

$$P(w_t | w_{t-1}) = \frac{\text{count}(w_{t-1}, w_t)}{\text{count}(w_{t-1})}$$

$$p(\text{The swift fox}) = P(\text{the} | \langle s \rangle) P(\text{swift} | \text{the}) P(\text{fox} | \text{swift})$$

Previous Word	Next Word	Probability
<s>	the	1.000
ancient	bridge	1.000
breeze	cools	1.000
bridge	</s>	1.000
cools	the	1.000
dog	</s>	1.000
evening	</s>	1.000
flows	under	1.000
fox	jumps	1.000
jumps	over	1.000
lazy	dog	1.000
over	the	1.000
river	flows	1.000
summer	evening	1.000
swift	breeze	0.333
swift	fox	0.333
swift	river	0.333
the	ancient	0.167
the	lazy	0.167
the	swift	0.500
the	warm	0.167
under	the	1.000
warm	summer	1.000

例子: 实现一个二元语言模型

语料库

The swift fox jumps over the lazy dog.

The swift river flows under the ancient bridge.

The swift breeze cools the warm summer evening.

语言模型

$$P(w_1, \dots, w_T) = \prod_{t=1}^T P(w_t | w_{t-1})$$

$$P(w_t | w_{t-1}) = \frac{\text{count}(w_{t-1}, w_t)}{\text{count}(w_{t-1})}$$

$$\begin{aligned} p(\text{The swift fox}) &= P(\text{the} | \langle s \rangle) P(\text{swift} | \text{the}) P(\text{fox} | \text{swift}) \\ &= 1.0 \times 0.5 \times 0.3333 = 0.1667 \end{aligned}$$

Previous Word	Next Word	Probability
<s>	the	1.000
ancient	bridge	1.000
breeze	cools	1.000
bridge	</s>	1.000
cools	the	1.000
dog	</s>	1.000
evening	</s>	1.000
flows	under	1.000
fox	jumps	1.000
jumps	over	1.000
lazy	dog	1.000
over	the	1.000
river	flows	1.000
summer	evening	1.000
swift	breeze	0.333
swift	fox	0.333
swift	river	0.333
the	ancient	0.167
the	lazy	0.167
the	swift	0.500
the	warm	0.167
under	the	1.000
warm	summer	1.000

例子: 实现一个二元语言模型

语料库

The swift fox jumps over the lazy dog.

The swift river flows under the ancient bridge.

The swift breeze cools the warm summer evening.

语言模型

$$P(w_1, \dots, w_T) = \prod_{t=1}^T P(w_t | w_{t-1})$$

$$P(w_t | w_{t-1}) = \frac{\text{count}(w_{t-1}, w_t)}{\text{count}(w_{t-1})}$$

P(I love this movie) = ?

Previous Word	Next Word	Probability
<s>	the	1.000
ancient	bridge	1.000
breeze	cools	1.000
bridge	</s>	1.000
cools	the	1.000
dog	</s>	1.000
evening	</s>	1.000
flows	under	1.000
fox	jumps	1.000
jumps	over	1.000
lazy	dog	1.000
over	the	1.000
river	flows	1.000
summer	evening	1.000
swift	breeze	0.333
swift	fox	0.333
swift	river	0.333
the	ancient	0.167
the	lazy	0.167
the	swift	0.500
the	warm	0.167
under	the	1.000
warm	summer	1.000

例子: 实现一个二元语言模型

语料库

The swift fox jumps over the lazy dog.

The swift river flows under the ancient bridge.

The swift breeze cools the warm summer evening.

语言模型

$$P(w_1, \dots, w_T) = \prod_{t=1}^T P(w_t | w_{t-1})$$

$$P(w_t | w_{t-1}) = \frac{\text{count}(w_{t-1}, w_t)}{\text{count}(w_{t-1})}$$

P(I love this movie)

= P(I|<s>) × P(love|I) × P(this|love) × P(movie|this) = 0

Previous Word	Next Word	Probability
<s>	the	1.000
ancient	bridge	1.000
breeze	cools	1.000
bridge	</s>	1.000
cools	the	1.000
dog	</s>	1.000
evening	</s>	1.000
flows	under	1.000
fox	jumps	1.000
jumps	over	1.000
lazy	dog	1.000
over	the	1.000
river	flows	1.000
summer	evening	1.000
swift	breeze	0.333
swift	fox	0.333
swift	river	0.333
the	ancient	0.167
the	lazy	0.167
the	swift	0.500
the	warm	0.167
under	the	1.000
warm	summer	1.000

基于N元的语言模型的问题

- 频率估计可靠性依赖于语料大小
- 包含没有见过的ngram的句子的概率为0

如何处理未登录词

- 构建词表的时候把低频词替换成<UNK>
- 计算的时候如果遇到未登录词当作<UNK>处理

如何处理未登录词

- 构建词表的时候把低频词替换成<UNK>
- 计算的时候如果遇到未登录词当作<UNK>处理

未登录词（Out-Of-Vocabulary words, 简称OOV），是指在自然语言处理（NLP）、信息检索、计算机词汇学等领域中，那些没有出现在现有词汇数据库、字典或索引中的词语

如何估计未登录词的频率

语言模型的平滑

- 在真实世界中，“没见过”是不是等于“不可能”？
- 只是 数据太少
- 所以这种情况下：给没见过的情況一点小概率

语言模型的平滑

平滑技术：是增加低频词的频率，而降低高频词的频率

$$\begin{aligned} P(w_t | w_{1:(t-1)}) &= P(w_t | w_{(t-n+1):(t-1)}) \\ &= \frac{\text{count}(w_{(t-n+1):t})}{\text{count}(w_{(t-n+1):(t-1)})} \\ &= \frac{\text{count}(w_{(t-n+1):t}) + \delta}{\text{count}(w_{(t-n+1):(t-1)}) + \delta|\mathcal{V}|} \end{aligned}$$

语言模型的平滑

$P(w \mid \text{denied the})$

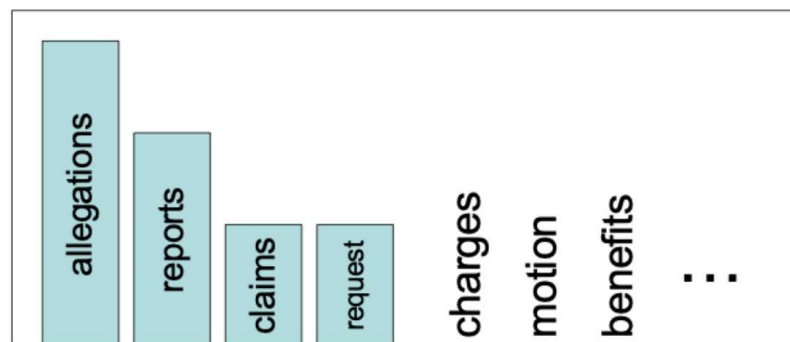
3 allegations

2 reports

1 claims

1 request

7 total



$P(w \mid \text{denied the})$

2.5 allegations

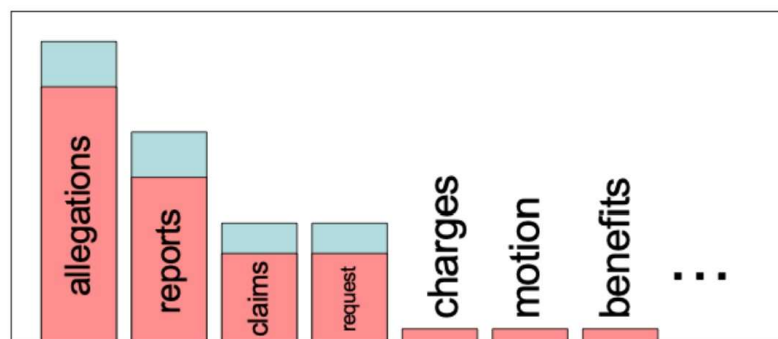
1.5 reports

0.5 claims

0.5 request

2 other

7 total



图来自于: Dan Klein and John DeNero

例子: 实现一个二元语言模型

语料库

The swift fox jumps over the lazy dog.

The swift river flows under the ancient bridge.

The swift breeze cools the warm summer evening.

语言模型

$$P(w_1, \dots, w_T) = \prod_{t=1}^T P(w_t | w_{t-1})$$

$$P(w_t | w_{t-1}) = \frac{\text{count}(w_{t-1}, w_t)}{\text{count}(w_{t-1})}$$

P(I love this movie)

= P(I | <s>) × P(love | I) × P(this | love) × P(movie | this) = 0

Previous Word	Next Word	Probability
<s>	the	1.000
ancient	bridge	1.000
breeze	cools	1.000
bridge	</s>	1.000
cools	the	1.000
dog	</s>	1.000
evening	</s>	1.000
flows	under	1.000
fox	jumps	1.000
jumps	over	1.000
lazy	dog	1.000
over	the	1.000
river	flows	1.000
summer	evening	1.000
swift	breeze	0.333
swift	fox	0.333
swift	river	0.333
the	ancient	0.167
the	lazy	0.167
the	swift	0.500
the	warm	0.167
under	the	1.000
warm	summer	1.000

二元语言模型+平滑

语料库

The swift fox jumps over the lazy dog.

The swift river flows under the ancient bridge.

The swift breeze cools the warm summer evening.

语料中的不同单词 : The, swift, fox, jumps, over, lazy, dog, river, flows, under, ancient, bridge, breeze, cools, warm, summer, evening。共17个词, 加上句子开始符 <s> 和结束符 </s>

共 :

$$|V| = 19$$

$P(\text{I love this movie})$

$= P(\text{I}|\text{<s>}) \times P(\text{love}|\text{I}) \times P(\text{this}|\text{love}) \times P(\text{movie}|\text{this})$

二元语言模型+平滑

语料库

The swift fox jumps over the lazy dog.

The swift river flows under the ancient bridge.

The swift breeze cools the warm summer evening.

语料中的不同单词 : The, swift, fox, jumps, over, lazy, dog, river, flows, under, ancient, bridge, breeze, cools, warm, summer, evening. 共17个词, 加上句子开始符 <s> 和结束符 </s>

共 :

$$|V| = 19$$

$P(I \text{ love this movie})$

$$= P(I | <s>) \times P(\text{love} | I) \times P(\text{this} | \text{love}) \times P(\text{movie} | \text{this})$$

$P(I | <s>)$

原始计数 :

$$\text{count}(<s>, I) = 0$$

$$\text{count}(<s>) = 3$$

(因为有 3 个句子)

套公式 :

$$P(I | <s>) = \frac{0 + 1}{3 + 19} = \frac{1}{22}$$

不再是 0 了。

二元语言模型+平滑

语料库

The swift fox jumps over the lazy dog.

The swift river flows under the ancient bridge.

The swift breeze cools the warm summer evening.

语料中的不同单词 : The, swift, fox, jumps, over, lazy, dog, river, flows, under, ancient, bridge, breeze, cools, warm, summer, evening. 共17个词, 加上句子开始符 <s> 和结束符 </s>

共 :

$$|V| = 19$$

$P(I \text{ love this movie})$

$$= P(I | <s>) \times P(\text{love} | I) \times P(\text{this} | \text{love}) \times P(\text{movie} | \text{this})$$

$P(\text{love} | I)$

因为 I 从未出现过 ,
所以 :

$$\text{count}(I) = 0$$

$$P(\text{love} | I) = \frac{0 + 1}{0 + 19} = \frac{1}{19}$$

二元语言模型+平滑

语料库

The swift fox jumps over the lazy dog.

The swift river flows under the ancient bridge.

The swift breeze cools the warm summer evening.

语料中的不同单词 : The, swift, fox, jumps, over, lazy, dog, river, flows, under, ancient, bridge, breeze, cools, warm, summer, evening. 共17个词, 加上句子开始符 <s> 和结束符 </s>

共 :

$$|V| = 19$$

$P(I \text{ love this movie})$

$= P(I|<s>) \times P(\text{love}|I) \times P(\text{this}|\text{love}) \times P(\text{movie}|\text{this})$

$P(\text{this}|\text{love}) = ?$

二元语言模型+平滑

语料库

The swift fox jumps over the lazy dog.

The swift river flows under the ancient bridge.

The swift breeze cools the warm summer evening.

语料中的不同单词 : The, swift, fox, jumps, over, lazy, dog, river, flows, under, ancient, bridge, breeze, cools, warm, summer, evening. 共17个词, 加上句子开始符 <s> 和结束符 </s>

共 :

$$|V| = 19$$

$P(\text{I love this movie})$

$$= P(\text{I}|\text{<s>}) \times P(\text{love}|\text{I}) \times P(\text{this}|\text{love}) \times P(\text{movie}|\text{this})$$

$P(\text{this}|\text{love})=?$

$P(\text{movie}|\text{this})=?$

二元语言模型+平滑

语料库

The swift fox jumps over the lazy dog.

The swift river flows under the ancient bridge.

The swift breeze cools the warm summer evening.

语料中的不同单词 : The, swift, fox, jumps, over, lazy, dog, river, flows, under, ancient, bridge, breeze, cools, warm, summer, evening. 共17个词, 加上句子开始符 <s> 和结束符 </s>

共 :

$$|V| = 19$$

$P(\text{I love this movie})$

$= P(\text{I} | \text{<s>}) \times P(\text{love} | \text{I}) \times P(\text{this} | \text{love}) \times P(\text{movie} | \text{this})$

$=$

神经网络语言模型

- 给每一个句子 (S) 都赋予其出现的概率。以 $w_1, w_2, \dots, w_T$ 这 T 个词组成的句子为例，它出现的概率为

$$\begin{aligned} P(S = w_{1:T}) &= P(w_1, \dots, w_T) \\ &= P(w_1)P(w_2|w_1)P(w_3|w_1w_2) \cdots P(w_T|w_{1:(T-1)}) \\ &= P(w_1) \prod_{t=2}^T P(w_t|w_{1:(t-1)}) \\ &= \prod_{t=1}^T P(w_t|w_{1:(t-1)}), \end{aligned}$$

句子中每个词 w_t 在给定前面词序列 $w_{1:(t-1)}$ 时的条件概率，如何变成一个分类问题？

神经网络语言模型

- 给每一个句子 (S) 都赋予其出现的概率。以 $w_1, w_2, \dots, w_T$ 这 T 个词组成的句子为例，它出现的概率为

$$\begin{aligned} P(S = w_{1:T}) &= P(w_1, \dots, w_T) \\ &= P(w_1)P(w_2|w_1)P(w_3|w_1w_2) \cdots P(w_T|w_{1:(T-1)}) \\ &= P(w_1) \prod_{t=2}^T P(w_t|w_{1:(t-1)}) \\ &= \prod_{t=2}^T \boxed{P(w_t|w_{1:(t-1)})}, \\ &P_\theta(v_k|\vec{w}_{1:(t-1)}) \end{aligned}$$

这个问题可以转换为一个类别数为 $|V|$ 的多类分类问题

词汇表 V 中的每个词 $v_k (1 \leq k \leq |V|)$ 出现的概率

$$= f_k(w_{1:(t-1)}, \theta),$$

神经网络语言模型

- 给每一个句子 (S) 都赋予其出现的概率。以 $w_1, w_2, \dots, w_T$ 这 T 个词组成的句子为例，它出现的概率为

$$\begin{aligned} P(S = w_{1:T}) &= P(w_1, \dots, w_T) \\ &= P(w_1)P(w_2|w_1)P(w_3|w_1w_2) \cdots P(w_T|w_{1:(T-1)}) \\ &= P(w_1) \prod_{t=2}^T P(w_t|w_{1:(t-1)}) \\ &= \prod_{t=1}^T P(w_t|w_{1:(t-1)}), \end{aligned}$$

这个问题可以转换为一个类别数为 $|V|$ 的多类分类问题

$$P_{\theta}(v_k|w_{1:(t-1)})$$

词汇表 V 中的每个词 $v_k (1 \leq k \leq |V|)$ 出现的概率

$$= f_k(w_{1:(t-1)}, \theta),$$

可以用不同分类器来估计语言模型的条件概率

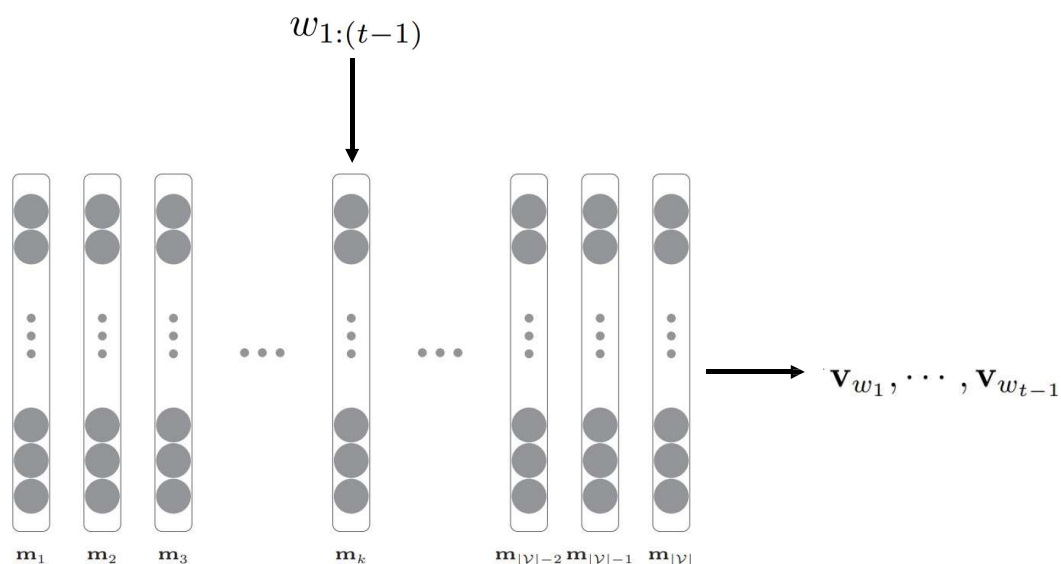
神经网络语言模型

- 输入层)
 - 功能：将离散的单词 $w_{1:(t-1)}$ 转化成向量表示
 - **词嵌入矩阵**：存储词表大小多个向量 ($M \in \mathbb{R}^{d_1 \times |\mathcal{V}|}$)
 - 查表： $\mathbf{v}_{w_i} = M\mathbf{e}_k = \mathbf{e}_k$

神经网络语言模型

□ 输入层

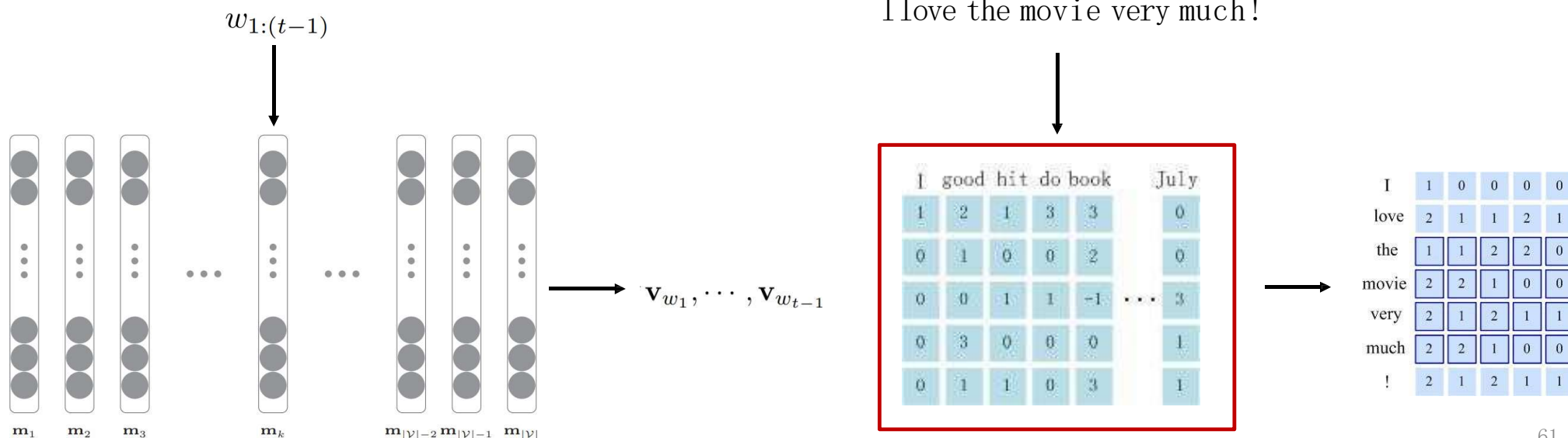
- 功能：将离散的单词 $w_{1:(t-1)}$ 转化成向量表示
- 词嵌入矩阵：存储词表大小多个向量 ($M \in \mathbb{R}^{d_1 \times |\mathcal{V}|}$)
- 查表： $\mathbf{v}_{w_i} = M\mathbf{e}_k = \mathbf{e}_k$



神经网络语言模型

□ 输入层

- 功能：将离散的单词 $w_{1:(t-1)}$ 转化成向量表示
- 词嵌入矩阵：存储词表大小多个向量 ($M \in \mathbb{R}^{d_1 \times |\mathcal{V}|}$)
- 查表： $\mathbf{v}_{w_i} = M\mathbf{e}_k = \mathbf{e}_k$



神经网络语言模型

□ 隐藏层

- 功能：总结输入的词向量为历史信息向量
- 输入： $\mathbf{v}_{w_1}, \dots, \mathbf{v}_{w_{t-1}}$
- 输出： $\mathbf{h}_t$

神经网络语言模型

□ 隐藏层

- 功能：总结输入的词向量为历史信息向量
- 输入： $\mathbf{v}_{w_1}, \dots, \mathbf{v}_{w_{t-1}}$
- 输出： $\mathbf{h}_t$

如何实现？

神经网络语言模型

□ 隐藏层

- 功能：总结输入的词向量为历史信息向量
- 输入： $\mathbf{v}_{w_1}, \dots, \mathbf{v}_{w_{t-1}}$
- 输出： $\mathbf{h}_t$
- 网络类型：

□ Bag-of-words

$$\mathbf{h}_t = \sum_1^{t-1} C_i \mathbf{v}_{w_i},$$

神经网络语言模型

□ 隐藏层

■ 功能：总结输入的词向量为历史信息向量

■ 输入： $\mathbf{v}_{w_1}, \dots, \mathbf{v}_{w_{t-1}}$

■ 输出： $\mathbf{h}_t$

■ 网络类型：

□ Bag-of-words

□ 前馈神经网络：

$$\mathbf{x}_t = \mathbf{v}_{w_{t-n+1}} \oplus \dots \oplus \mathbf{v}_{w_{t-1}}$$

$$\mathbf{h}_t = \tanh(W\mathbf{x}_t + \mathbf{c})$$

神经网络语言模型

□ 隐藏层

■ 功能：总结输入的词向量为历史信息向量

■ 输入： $\mathbf{v}_{w_1}, \dots, \mathbf{v}_{w_{t-1}}$

■ 输出： $\mathbf{h}_t$

■ 网络类型：

□ Bag-of-words

□ 前馈神经网络

□ 循环神经网络

$$\mathbf{h}_t = \tanh(U\mathbf{h}_{t-1} + W\mathbf{v}_{w_{t-1}} + \mathbf{c}),$$

神经网络语言模型

□ 输出层

- 大小: 输出层大小为 $|V|$
- 输入: 历史信息表示向量 $\mathbf{h}_t \in \mathbb{R}^{d_2}$
- 输出: 词表大小的概率分布 $\mathbf{y}_t \in \mathbb{R}^{|V|}$

神经网络语言模型

□ 输出层

- 大小：输出层大小为 $|V|$
- 输入：历史信息表示向量 $\mathbf{h}_t \in \mathbb{R}^{d_2}$
- 输出：词表大小的概率分布 $\mathbf{y}_t \in \mathbb{R}^{|V|}$

$$\mathbf{y}_t = \text{softmax}(\mathbf{O}\mathbf{h}_t + \mathbf{b}),$$

输出词嵌入矩阵

$$\begin{aligned} P_{\theta}(v_k|h_t) &= [\mathbf{y}_t]_k \\ &= \text{softmax}(s(v_k, h_t; \theta)) \\ &= \frac{\exp(s(v_k, h_t; \theta))}{\sum_{j=1}^{|V|} \exp(s(v_j, h_t; \theta))}, \end{aligned}$$

其中：

$$s(v_k, h; \theta) = \mathbf{o}_k^T \mathbf{h} + b_k$$

神经网络语言模型

□ 输出层

- 大小：输出层大小为 $|V|$
- 输入：历史信息表示向量 $\mathbf{h}_t \in \mathbb{R}^{d_2}$
- 输出：词表大小的概率分布 $\mathbf{y}_t \in \mathbb{R}^{|V|}$

$$\mathbf{y}_t = \text{softmax}(\mathbf{O}\mathbf{h}_t + \mathbf{b}),$$

输出词嵌入矩阵

□ 输入层

- 功能：将离散的单词 $w_{1:(t-1)}$ 转化成向量表示
- 词嵌入矩阵：存储词表大小多个向量 ($\mathbf{M} \in \mathbb{R}^{d_1 \times |V|}$)
- 查表： $\mathbf{v}_{w_t} = \mathbf{M}\mathbf{e}_k = \mathbf{v}_k$

$$\begin{aligned} P_{\theta}(v_k|h_t) &= [\mathbf{y}_t]_k \\ &= \text{softmax}(s(v_k, h_t; \theta)) \\ &= \frac{\exp(s(v_k, h_t; \theta))}{\sum_{j=1}^{|V|} \exp(s(v_j, h_t; \theta))}, \end{aligned}$$

其中：

$$s(v_k, h; \theta) = \mathbf{o}_k^T \mathbf{h} + b_k$$

实例：手算一个神经网络语言模型

- 词表 $V = \{I, \text{love}, \text{NLP}, \text{deep}, \text{learning}\}$

- 编号：

- $1 \rightarrow I$

- $2 \rightarrow \text{love}$

- $3 \rightarrow \text{NLP}$

- $4 \rightarrow \text{deep}$

- $5 \rightarrow \text{learning}$

- 词表大小 $|V| = 5$

词嵌入矩阵 M (2维)

$$M = \begin{bmatrix} 1 & 0 & 1 & 2 & 2 \\ 0 & 1 & 1 & 1 & 2 \end{bmatrix}$$

对应：

- $I \rightarrow (1,0)$

- $\text{love} \rightarrow (0,1)$

- $\text{NLP} \rightarrow (1,1)$

- $\text{deep} \rightarrow (2,1)$

- $\text{learning} \rightarrow (2,2)$

实例：手算一个神经网络语言模型

- 当前输入："I love"我们预测下一个词。

句子表示计算方式：

第一步：把词向量拼接 (concatenate)

$$x = \begin{bmatrix} v_I \\ v_{love} \end{bmatrix}$$

第二步：通过全连接层：

$$h = xW + b$$

其中：

$$W = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$$

$$b = (0, 0)$$

实例：手算一个神经网络语言模型

- 输出权重矩阵：

$$O = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \\ 2 & 1 \\ 1 & 2 \end{bmatrix}$$

偏置为0。

$$s = Oh$$
$$P(word) = softmax(s)$$

实例：手算一个神经网络语言模型

- 请完成：
 1. 写出 I 和 love 的词向量
 2. 计算拼接向量 x
 3. 计算隐藏层 h
 4. 计算输出层打分 s
 5. 计算 softmax 概率
 6. 预测下一个词

作业-1

- 输入句子：
- "I deep"
- 请完成：
 1. 写出 I 和 deep 的词向量
 2. 计算拼接向量 x
 3. 计算隐藏层 h
 4. 计算输出层打分 s
 5. 计算 softmax 概率
 6. 预测下一个词



谢谢！ 欢迎提问！

Instructor: Xiaohan Ma(马晓寒), Ph.D.

Graduate School of Information and Communication Technology,

Ajou University, Korea

maxiaohan@ajou.ac.kr